

SKILL SESSION 3&4

Session 3:

TASK 1:

UW PICO 5.09

File: history.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_CMD 100

char history[MAX_HISTORY][MAX_CMD];
int history_count = 0;
int current = -1;

void add_history(char *cmd) {
    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], cmd);
        history_count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
            strcpy(history[i], history[i + 1]);

        strcpy(history[MAX_HISTORY - 1], cmd);
    }

    current = history_count;
}

void show_history() {
    printf("\nCommand History:\n");

    for (int i = 0; i < history_count; i++)
        printf("%d : %s\n", i + 1, history[i]);
}

void previous_command() {
    if (history_count == 0) {
        printf("No commands in history.\n");
        return;
    }

    if (current > 0)
        current--;

    printf("Previous command: %s\n", history[current]);
}
```

```
        printf("No commands in history.\n");
        return;
    }

    if (current < history_count - 1)
        current++;

    printf("Next command: %s\n", history[current]);
}

int main() {
    char command[MAX_CMD];

    printf("Command History System\n");

    while (1) {
        printf("\nEnter command: ");
        fgets(command, MAX_CMD, stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            show_history();
        }
        else if (strcmp(command, "prev") == 0) {
            previous_command();
        }
        else if (strcmp(command, "next") == 0) {
            next_command();
        }
        else {
            add_history(command);
            printf("Command stored.\n");
        }
    }

    return 0;
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air ~ % mkdir Skill
mkdir: Skill: File exists
[(base) saiabburi@Geyasree-MacBook-Air ~ % cd Skill
[(base) saiabburi@Geyasree-MacBook-Air Skill % nano history.c
```

```
(base) saiabburi@Geyasree-MacBook-Air Skill % gcc history.c -o history
./history
Command History System
```

```
Enter command: ls
Command stored.
```

```
Enter command: pwd
Command stored.
```

```
Enter command: gcc test.c
Command stored.
```

```
Enter command: history
```

```
Command History:
```

```
1 : ls
2 : pwd
3 : gcc test.c
```

```
Enter command: prev
Previous command: gcc test.c
```

```
Enter command: prev
Previous command: pwd
```

```
Enter command: next
Next command: gcc test.c
```

```
Enter command: exit
```

TASK 2:

UW PICO 5.09

File: buffer.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void insert(Node **head, int value) {
    Node *newNode = malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = *head;
    *head = newNode;
}

void display(Node *head) {
    printf("Linked List: ");

    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }

    printf("NULL\n");
}

void freeList(Node *head) {
    Node *temp;

    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    int size = 2;
    int count = 0;
    int value;
```

```

int *buffer = malloc(size * sizeof(int));

if (buffer == NULL) {
    printf("Memory allocation failed.\n");
    return 1;
}

printf("Dynamic Buffer Allocation\n");

for (int i = 0; i < 5; i++) {
    printf("Enter value: ");
    scanf("%d", &value);

    if (count == size) {
        size *= 2;

        buffer = realloc(buffer, size * sizeof(int));

        if (buffer == NULL) {
            printf("Reallocation failed.\n");
            return 1;
        }

        printf("Buffer resized to %d elements.\n", size);
    }

    buffer[count++] = value;
}

printf("\nStored values: ");

for (int i = 0; i < count; i++)
    printf("%d ", buffer[i]);

printf("\n");

Node *head = NULL;

for (int i = 0; i < count; i++)
    insert(&head, buffer[i]);

display(head);

```

```

        return 0;
    }

    printf("\n");

    Node *head = NULL;

    for (int i = 0; i < count; i++)
        insert(&head, buffer[i]);

    display(head);

    free(buffer);
    freeList(head);

    printf("Memory released successfully.\n");

    return 0;
}

```

```

[(base) saiabburi@Geyasree-MacBook-Air Skill % nano buffer.c
(base) saiabburi@Geyasree-MacBook-Air Skill % gcc buffer.c -o buffer
./buffer
Dynamic Buffer Allocation
Enter value: 10
Enter value: 20
Enter value: 30
Buffer resized to 4 elements.
Enter value: 40
Enter value: 50
Buffer resized to 8 elements.

Stored values: 10 20 30 40 50
Linked List: 50 -> 40 -> 30 -> 20 -> 10 -> NULL
Memory released successfully.
[(base) saiabburi@Geyasree-MacBook-Air Skill % valgrind --version
valgrind (Ubuntu 8.0-2ubuntu1) 3.12.0
Copyright (C) 2013-2014 Julian Seward.  All rights reserved.
This program is free software; you can redistribute it and/or
modify it under the terms of the GNU General Public License
as published by the Free Software Foundation; either version 2
of the License, or (at your option) any later version.
This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
GNU General Public License for more details.
You should have received a copy of the GNU General Public License
along with this program; if not, write to the Free Software
Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA
02110-1301, USA.

```

Session 4:

TASK 1:

```
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 50
#define MAX_LENGTH 100

int main() {
    char input[MAX_LENGTH];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("Enter an expression: ");
    fgets(input, MAX_LENGTH, stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_TOKENS) {
        tokens[count] = token;
        count++;

        token = strtok(NULL, " \t\n");
    }

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i]);
    }

    printf("\nTotal tokens: %d\n", count);

    return 0;
}
```

```

((base) saiabburi@Geyasree-MacBook-Air ~ % nano tokenizer.c
((base) saiabburi@Geyasree-MacBook-Air ~ % gcc tokenizer.c -o tokenizer
((base) saiabburi@Geyasree-MacBook-Air ~ % ./tokenizer
Enter an expression: add 10 + 20

Tokens:
Token 1: add
Token 2: 10
Token 3: +
Token 4: 20

Total tokens: 4

```

TASK 2:

UW PICO 5.09

File: parser.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

int isNumber(char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        if (!isdigit(str[i])) {
            return 0;
        }
    }
    return 1;
}

int main() {
    char input[100];
    char *tokens[10];
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t");

    while (token != NULL && count < 10) {
        tokens[count] = token;
        count++;

        token = strtok(NULL, " \t");
    }

    /* Check for empty command */
    if (count == 0) {
        printf("Error: Empty command.\n");
        return 0;
    }

    /* Check ADD command */
    if (strcmp(tokens[0], "add") == 0) {

```



```

/* Check ADD command */
if (strcmp(tokens[0], "add") == 0) {

    if (count != 3) {
        printf("Syntax Error: add requires two numbers.\n");
        return 0;
    }

    if (!isNumber(tokens[1]) || !isNumber(tokens[2])) {
        printf("Syntax Error: operands must be numbers.\n");
        return 0;
    }

    int a = atoi(tokens[1]);
    int b = atoi(tokens[2]);

    printf("Valid command.\n");
    printf("Result: %d\n", a + b);
}

else {
    printf("Error: Unknown command.\n");
}

return 0;
}

```

```

[(base) saiabburi@Geyasree-MacBook-Air ~ % nano parser.c
[(base) saiabburi@Geyasree-MacBook-Air ~ % gcc parser.c -o parser
[(base) saiabburi@Geyasree-MacBook-Air ~ % ./parser
Enter command: add 10 20
Valid command.
Result: 30
[(base) saiabburi@Geyasree-MacBook-Air ~ % ./parser
Enter command: add 10
Syntax Error: add requires two numbers.
[(base) saiabburi@Geyasree-MacBook-Air ~ % ./parser
Enter command: add hello 20
Syntax Error: operands must be numbers.
[(base) saiabburi@Geyasree-MacBook-Air ~ % ./parser
Enter command:
Error: Empty command.
(base) saiabburi@Geyasree-MacBook-Air ~ % █

```

PRACTICE PROBLEMS

1. TASK 1:

```
[(base) saiabburi@Geyasree-MacBook-Air ~ % cd ~/OSSP
[(base) saiabburi@Geyasree-MacBook-Air OSSP % nano fork_exec.c
```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP % gcc fork_exec.c -o fork_exec
[(base) saiabburi@Geyasree-MacBook-Air OSSP % ./fork_exec
Enter a Linux command: ls

=== Parent Process ===
Parent PID : 3819
Child PID : 3832

=== Child Process ===
Child PID : 3832
Parent PID : 3819
Executing command: ls

LICENSE      README.md    bin          fork_exec    include      scripts      tests
Makefile     assets       docs         fork_exec.c  obj          src

Child process completed.
[(base) saiabburi@Geyasree-MacBook-Air OSSP %
```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP % tree
```

```
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── fork_exec
├── fork_exec.c
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c
```

```
9 directories, 20 files
```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP %
```

2. Task 1:

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP % man fork
```

```
Unknown locale, assuming C
```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP % man exec
```

```
Unknown locale, assuming C
```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP %
```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP % man execvp
```

```
Unknown locale, assuming C
```

Task 2:

```
/ directories, 20 files  
[(base) saiabburi@Geyasree-MacBook-Air OSSP % nano src/main.c
```

```
UW PICO 5.09
```

```
#include <stdio.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
#include <string.h>  
  
int main()  
{  
    pid_t pid;  
    char command[100];  
  
    printf("Enter a Linux command: ");  
    scanf("%s", command);  
  
    pid = fork();  
  
    if(pid == 0)  
    {  
        printf("\n=== Child Process ===\n");  
        printf("Child PID : %d\n", getpid());  
        printf("Parent PID : %d\n", getppid());  
        printf("Executing command: %s\n\n", command);  
  
        execlp(command, command, NULL);  
  
        printf("Command execution failed.\n");  
    }  
    else if(pid > 0)  
    {  
        printf("\n=== Parent Process ===\n");  
        printf("Parent PID : %d\n", getpid());  
        printf("Child PID : %d\n", pid);  
  
        wait(NULL);  
  
        printf("\nChild process completed.\n");  
    }  
    else  
    {  
        printf("Fork failed!\n");  
    }  
  
    return 0;  
}
```

```
[(base) saiabburi@Geyasree-MacBook-Air OSSP % ls bin
my_shell
[(base) saiabburi@Geyasree-MacBook-Air OSSP % ls
LICENSE          assets          fork_exec        obj              tests
Makefile         bin            fork_exec.c      scripts
README.md        docs          include          src
[(base) saiabburi@Geyasree-MacBook-Air OSSP % ./bin/my_shell
Enter a Linux command: ls

=== Parent Process ===
Parent PID : 5678
Child PID : 5693

=== Child Process ===
Child PID : 5693
Parent PID : 5678
Executing command: ls

LICENSE          assets          fork_exec        obj              tests
Makefile         bin            fork_exec.c      scripts
README.md        docs          include          src

Child process completed.
```